

LOW LEVEL DESIGN

The LLD Runbook

A working engineer's guide to the design interview.

LLD is the round where months of preparation can quietly go in the wrong direction. You can know every design pattern by name and still lose the room, because the round isn't testing recall. It's testing whether your design survives the next question.

This is the shortest honest path I know from *I understand OOP* to *I can defend a design under pressure*. Part 1 builds the vocabulary. Part 2 puts it to work on real problems.

Contents

Every numbered section is exactly one page.

1	What LLD actually is	Orientation
1.1	What LLD actually is	4
1.2	Four rounds that get confused	5
1.3	What's actually being evaluated	6
2	Object orientation, reconsidered	What interviewers probe
2.1	Encapsulation	7
2.2	Abstraction	8
2.3	Inheritance	9
2.4	Polymorphism	10
3	SOLID, with the bad code shown first	Before and after
3.1	Single Responsibility	11
3.2	Open/Closed	12
3.3	Liskov Substitution	13
3.4	Interface Segregation	14
3.5	Dependency Inversion	15
4	Principles that keep designs small	Restraint
4.1	KISS, DRY and YAGNI	16
4.2	Composition over inheritance	17
4.3	Interfaces and immutability	18
5	The patterns that actually come up	Six that carry most interviews
5.1	Choosing a pattern	19
5.2	Strategy	20
5.3	Factory	21
5.4	Observer	22
5.5	Singleton	23
5.6	Builder and Decorator	24
5.7	Runners-up and the rest	25
6	Putting it together	The method
6.1	A framework for any LLD problem	26

How to use this runbook

This book is short on purpose. Not because low level design is a small subject, but because the version of it that matters in an interview is much smaller than the literature around it.

How it's built

Part 1 is theory, and it is deliberately compressed — principles, patterns and a repeatable framework, with just enough code to make each idea concrete. Part 2 is where the real work happens: ten problems worked end to end, two pages each, in the same order you'd actually solve them in a room.

Read Part 1 once, cover to cover. Come back to it only through Part 2, when a problem makes you reach for something and you want to check that you're reaching for the right thing. Theory read on its own tends not to stick; theory read because a problem demanded it usually does.

I ASSUME YOU ALREADY KNOW

- A language with classes and interfaces — Java throughout, but nothing here is Java-specific.
- Basic data structures, enough to reason about a map or a linked list without looking them up.
- What an interface is, even if you've never thought hard about when to introduce one.

I DELIBERATELY SKIP

- Language syntax and OOP mechanics you can look up in a minute.
- The full pattern catalogue. Six patterns carry most interviews; the rest get a line each.
- Distributed systems concerns. That's a different round, and mixing the two costs candidates marks.

Three ways to read this

YOU HAVE	READ THIS
A few weeks	Part 1 cover to cover, then one problem from Part 2 each day. Attempt the design yourself before reading the worked answer.
A weekend	Sections 1.3, all of chapter 3, then 5.1 and 6.1. Follow with the LRU cache and HashMap problems.
Tonight	Sections 1.3, 5.1 and 6.1, then the pattern cheat sheet at the back of Part 2. That is the irreducible core.

A note on the code

Snippets here are short on purpose — usually the ten or so lines where the decision actually lives, with the scaffolding stripped out. Nobody learns anything from a full class listing printed on a page. Where a problem has a complete implementation worth reading, you'll find a link beside it:

github.com/varunkashyap-vkd/lld-runbook

All of it compiles and runs. If you learn by breaking things, clone it first and read the chapter second.

1.1 What LLD actually is

Ask ten engineers to define low level design and you'll get ten answers, most of them gesturing vaguely at classes and UML. That vagueness is the first thing working against you, so it's worth fixing before anything else.

A definition worth keeping

Low level design is turning a loosely worded requirement into classes, responsibilities and relationships a team could build on — and then defending those choices when the requirement changes.

Notice the two halves. The first is modelling. The second is defending. Almost everyone prepares for the first, and then loses the round in the second, because the interviewer's real question was never *can you draw this*. It was *what happens to your design when I change my mind*.

Where it shows up

- **A dedicated LLD round.** Forty-five to sixty minutes, a deliberately open prompt — design a parking lot, a rate limiter, a food delivery app. Code is usually optional; structure and reasoning are not.
- **A machine coding round.** Same modelling problem, but you ship running code in ninety minutes to three hours. The design bar is lower, the execution bar is much higher.
- **The last ten minutes of a DSA round.** Your solution works, and then the interviewer says *now make the eviction policy configurable*. The round just turned into LLD without anyone announcing it.
- **A hiring manager or bar-raiser conversation.** No code at all. You talk through a design you've built, and every trade-off you glossed over gets probed.

WORTH KNOWING

The third one catches the most people out. Candidates relax once the algorithm passes, and treat the follow-up as a bonus question rather than the part that's actually being scored. It's usually the part that's actually being scored.

What changes with your level

LEVEL	WHAT THEY NEED TO SEE	WHAT SINKS YOU
Entry	A correct, readable class structure. Sensible names, responsibilities in roughly the right place.	Procedural code wearing a class costume — one manager object doing everything.
Mid	Justified trade-offs. You chose composition here and an interface there, and can say why out loud.	Design on autopilot. Patterns applied because they're familiar, not because the problem asked.
Senior	Questioning the requirement itself, and drawing boundaries that hold when the follow-up lands.	Answering exactly what was asked, and nothing more.

1.2 Four rounds that get confused

DSA, LLD, machine coding and HLD blur together in most preparation material, and the blur is expensive. Knowing which room you're in changes what a good answer looks like.

	DSA	LLD	MACHINE CODING	HLD
Unit of work	An algorithm	A class model	A running module	Services and data flow
You optimise for	Correctness, then complexity	Structure that absorbs change	Working software, quickly	Behaviour at scale
Really scored on	Does it work, and how fast	Why these classes, and not others	Does it run, and is it clean	Trade-offs across the system
Typical length	45 min	45–60 min	90–180 min	45–60 min
Classic failure	Brute force, never optimised	Writing code before naming the nouns	Over-engineering until nothing runs	Naming technologies instead of reasoning

The gear shift that trips people up

DSA trains a specific instinct: there is one best answer, and your job is to find it. That instinct actively hurts you in an LLD round, where there is a family of defensible answers and the interviewer is watching how you choose between them. Two candidates can produce different class diagrams for the same prompt and both clear the bar — the one who can't explain their choice is the one who doesn't.

The practical consequence is about silence. In a DSA round, going quiet for two minutes while you think is normal, sometimes even a good sign. In an LLD round it's close to fatal, because the reasoning *is* the deliverable. A diagram that appears on the board without narration gives the interviewer nothing to score.

THE LINE BETWEEN LLD AND HLD

If your answer involves load balancers, replication or a message queue, you've drifted into HLD. If it involves which class owns a field and who is allowed to mutate it, you're in LLD. Drifting upward feels impressive and reads as avoidance — it's the most common way experienced candidates lose points on a problem they could have modelled well.

When you can't tell which room you're in

Ask. *Would you like me to focus on the class design, or do you want something running by the end?* is a perfectly reasonable opening question, and I've never seen it land badly. Recruiters describe rounds inconsistently, interviewers adapt on the day, and the thirty seconds you spend clarifying is far cheaper than twenty minutes spent solving the wrong problem well.

1.3 What's actually being evaluated

Interviewers rarely score the diagram. They score the sequence of decisions that produced it, and there are five of them worth preparing for directly.

The five things on the scorecard

1. **Requirement clarification.** Did you find the ambiguity before you built on top of it? Prompts are underspecified on purpose.
2. **Entity modelling.** Are your nouns the right nouns? Most bad designs are traceable to a class that should never have existed, or one that should have.
3. **Boundaries and abstraction.** What is hidden behind what, and can a caller do damage you didn't intend?
4. **Extensibility under pressure.** The follow-up change — how much of your design has to be rewritten to absorb it?
5. **Communication.** Whether the four above were visible to the person in the room, or happened silently in your head.

How strong DSA candidates lose this round

- **Coding too early.** Ten minutes of unprompted code buys you a structure you now feel obliged to defend.
- **Pattern-dropping.** A Factory, a Singleton and an Observer on a problem that needed one interface. It reads as insecurity, not fluency.
- **Designing for imaginary scale.** Thread pools and caches for a problem with three entities and no stated load.
- **Going quiet.** Covered on the previous page, and worth repeating — unnarrated thinking is unscored thinking.
- **Treating the follow-up as an attack.** Defensiveness here is a bad look. The change was always coming; it's the whole point of the round.

What good sounds like

The gap between a pass and a strong pass is usually one sentence pattern, repeated all the way through: **state the choice, name the alternative, give the reason.**

"I'll put eviction behind an interface rather than a flag on the cache. The flag is less code today, but every new policy means editing the same class — and I'd rather add a file than edit one."

That takes twenty seconds and covers three of the five items above. Say it out loud enough times in practice and it stops sounding rehearsed.

IF YOU TAKE ONE THING FROM THIS CHAPTER

The follow-up isn't a curveball, and it isn't the interviewer being difficult. It's the exam. Everything before it exists to set up the moment where the requirement shifts and we find out whether your design bends or breaks. Chapter 6 turns that into a repeatable sequence you can run on any prompt.

2.1 Encapsulation

Encapsulation is not "make fields private and add getters". It is deciding who is allowed to change state, and then refusing everyone else.

The useful question is never *what is the visibility of this field*. It is *what does this class promise about itself, and could a caller break that promise?* A cart whose total can disagree with its items is broken no matter how private the fields are.

ANTI-PATTERN

```
class Cart {  
    public List<Item> items = new ArrayList<>();  
    public Money total;           // who keeps this in sync?  
}
```

BETTER

```
class Cart {  
    private final List<Item> items = new ArrayList<>();  
  
    void add(Item item) { items.add(item); }  
  
    List<Item> items() { return List.copyOf(items); } // no outside mutation  
  
    Money total() {                               // derived, never stale  
        return items.stream().map(Item::price)  
            .reduce(Money.ZERO, Money::add);  
    }  
}
```

Two decisions did the work. The total is computed rather than stored, so there is no second copy of the truth to fall out of step. And the list is handed out as a copy, so a caller holding a reference cannot quietly add to it.

Common mistakes

- **Getter and setter for every field.** A private field with a public setter is a public field with extra typing. Ask what the setter protects.
- **Returning the internal collection.** The most common one. It hands every caller a key to your state.
- **Validating in the caller.** If the object can be constructed in an invalid state, someone eventually will.

WHAT THE INTERVIEWER IS CHECKING

Whether you can name the invariant out loud. "A cart's total must always equal the sum of its items" is the sentence that justifies every access decision on the page. Say it and the design explains itself; skip it and you are just guessing at keywords.

2.2 Abstraction

Encapsulation hides state. Abstraction hides decisions — it stops callers from knowing *how* something is done, so you can change your mind later.

ANTI-PATTERN

```
class OrderService {
    void place(Order order) {
        SmtplibClient smtp = new SmtplibClient("smtp.internal", 587);
        smtp.send(order.customerEmail(), "Order placed");
    }
}
```

Nothing here is wrong until the day you add SMS. Then `OrderService` — a class about orders — has to be edited because of a messaging decision. It knows a hostname and a port number, and neither has anything to do with placing an order.

BETTER

```
interface Notifier {
    void notify(Customer to, String message);
}

class OrderService {
    private final Notifier notifier;

    OrderService(Notifier notifier) { this.notifier = notifier; }

    void place(Order order) {
        notifier.notify(order.customer(), "Order placed");
    }
}
```

How a leaking abstraction announces itself

Watch the names. The moment a technology appears in the contract — `sendEmail()`, `getConnection()`, `toJson()` — the abstraction has failed, because every caller now depends on the choice you were trying to hide.

Interface or abstract class

	USE IT WHEN	COST
Interface	You need a contract and nothing more. This is the default.	No shared code; each implementation stands alone.
Abstract class	Subclasses genuinely share state or a fixed algorithm skeleton.	Spends the one inheritance slot and couples subclasses to the base.

Common mistakes

- **An interface with one implementation, added "for flexibility".** If no second implementation is coming, that is indirection, not abstraction.
- **Methods only one implementation can honour.** The rest throw, and the contract was a lie.

2.3 Inheritance

Inheritance is the most over-used relationship in LLD interviews. It buys you code reuse and charges you coupling, and candidates rarely price the second half.

ANTI-PATTERN

```
class Stack extends ArrayList<String> {
    void push(String s) { add(s); }
    String pop() { return remove(size() - 1); }
}
```

This compiles, passes a quick test, and is broken. `Stack` did not inherit two useful methods — it inherited the *entire* `ArrayList` API. A caller can now write `stack.add(0, "x")` and insert at the bottom, or call `clear()`, or index into the middle. The one thing a stack promises is the one thing this class cannot enforce.

BETTER

```
class Stack {
    private final List<String> items = new ArrayList<>();

    void push(String s) { items.add(s); }

    String pop() {
        if (items.isEmpty()) throw new NoSuchElementException();
        return items.remove(items.size() - 1);
    }
}
```

Same reuse, none of the exposure. The list is now a private implementation detail rather than a public inheritance contract.

When inheritance is genuinely right

All three have to hold, not just the first:

- The relationship is a real *is-a*, and callers will substitute the subtype for the base.
- The base was designed for extension — stable, documented, and yours to change.
- The subclass weakens no guarantee the base makes.

Common mistakes

- **Inheriting for reuse rather than substitutability.** If you never treat the subtype as the base, you wanted a field, not a parent.
- **Overriding a method to throw.** `UnsupportedOperationException` is an admission that the hierarchy is wrong.

WHAT THE INTERVIEWER IS CHECKING

Whether `extends` is your reflex. Reaching for composition first, and giving a reason when you do inherit, separates people who learned OOP from people who have maintained it. If you inherit, say which guarantee the base makes and why your subclass keeps it.

2.4 Polymorphism

Textbooks explain polymorphism with animals making sounds. In an interview its value is far more concrete: it is how you delete a conditional that would otherwise grow forever.

ANTI-PATTERN

```
Money fee(Payment p) {
    switch (p.type()) {
        case CARD:    return p.amount().percent(2);
        case UPI:     return Money.ZERO;
        case WALLET:  return p.amount().percent(1);
    }
    throw new IllegalArgumentException(p.type().name());
}
```

The problem is not this method. It is that a codebase like this never has only one — there is a matching switch for settlement time, another for refund rules, another for the receipt label. Adding a payment type means finding all of them, and the compiler will not tell you when you miss one.

BETTER

```
interface PaymentMethod {
    Money fee(Money amount);
}

class Card implements PaymentMethod {
    public Money fee(Money amount) { return amount.percent(2); }
}

class Upi implements PaymentMethod {
    public Money fee(Money amount) { return Money.ZERO; }
}
```

Adding a wallet is now a new file rather than an edit to four existing ones. That is the Open/Closed Principle arriving early — we come back to it in 3.2, and it is the backbone of the Strategy pattern in 5.2.

When a switch is the right answer

On a closed set that genuinely will not grow — days of the week, HTTP methods — a switch is clearer than four classes. Replacing every conditional with a type hierarchy is its own failure mode, and an experienced interviewer will read it as over-engineering.

Common mistakes

- **instanceof chains.** The same growing conditional wearing a disguise. If you are asking what type something is, the type should be deciding.
- **Logic living in the caller.** Data-only classes with all the behaviour outside them means polymorphism has nowhere to happen.